

Monte Carlo Analysis of IDW Interpolation Error on Meuse Dataset - CA1

Gaurav Padalkar, 202301126001, TY-Mechatronics, JNEC

Introduction

This study evaluates the performance of Inverse Distance Weighting (IDW) interpolation on the Meuse dataset using a Monte Carlo simulation framework.

The simulation was performed externally, and results were stored in a database. After that, a separate analysis code pulled the errors from the database and plotted them as required.

Methodology

A Monte Carlo approach was used to assess interpolation uncertainty:

- Random interior test points were generated using a Point-in-Polygon (PiP) method.
- Gaussian noise (10% of standard deviation) was added to both training and test data.
- IDW interpolation (power = 2) was applied repeatedly.
- Errors were computed as the difference between predicted and noisy true values.
- Results were stored in an SQLite database for efficient retrieval and analysis.

IDW Computation (Reference Implementation)

```
library(sp)
library(gstat)

data(meuse, package = "sp")

meuse_og_df <- meuse
coordinates(meuse) <- ~x + y

coords <- coordinates(meuse)
ch_idx <- chull(coords)
ch_coords <- coords[c(ch_idx, ch_idx[1]), ]

xmin <- min(ch_coords[,1])
xmax <- max(ch_coords[,1])
ymin <- min(ch_coords[,2])
ymax <- max(ch_coords[,2])

set.seed(69)
```

```

n_test <- 3
inside_points <- matrix(NA, n_test, 2)

count <- 0
while (count < n_test) {
  x_rand <- runif(1, xmin, xmax)
  y_rand <- runif(1, ymin, ymax)

  if (point.in.polygon(x_rand, y_rand,
                       ch_coords[,1], ch_coords[,2]) == 1) {
    count <- count + 1
    inside_points[count, ] <- c(x_rand, y_rand)
  }
}

inside_sp <- SpatialPoints(
  inside_points,
  proj4string = CRS(proj4string(meuse))
)

true_field <- idw(zinc ~ 1, meuse, inside_sp, idp = 2)
z_true <- true_field$var1.pred

n_sim <- 1000
sigma <- sd(meuse_og_df$zinc) * 0.1

noise_train <- matrix(rnorm(nrow(meuse)*n_sim, sd=sigma), nrow=nrow(meuse))
noise_test  <- matrix(rnorm(n_test*n_sim, sd=sigma), nrow=n_test)

pred <- matrix(NA, n_test, n_sim)
truth <- matrix(NA, n_test, n_sim)
error <- matrix(NA, n_test, n_sim)

for (i in 1:n_sim) {
  meuse$z_noisy <- meuse_og_df$zinc + noise_train[, i]

  idw_pred <- idw(z_noisy ~ 1, meuse, inside_sp, idp = 2)

  pred[, i] <- idw_pred$var1.pred
  truth[, i] <- z_true + noise_test[, i]
  error[, i] <- pred[, i] - truth[, i]
}

```

Data Retrieval

```

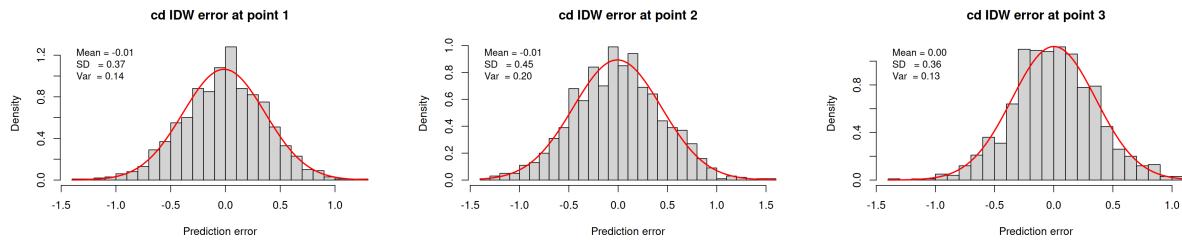
library(DBI)
library(RSQLite)

conn <- dbConnect(SQLite(), "idw_errors.db")
df_errors <- dbReadTable(conn, "idw_errors")

```

```
df_errors_p1 <- df_errors[df_errors$test_point == 1, ]
df_errors_p2 <- df_errors[df_errors$test_point == 2, ]
df_errors_p3 <- df_errors[df_errors$test_point == 3, ]
```

Cadmium (Cd)

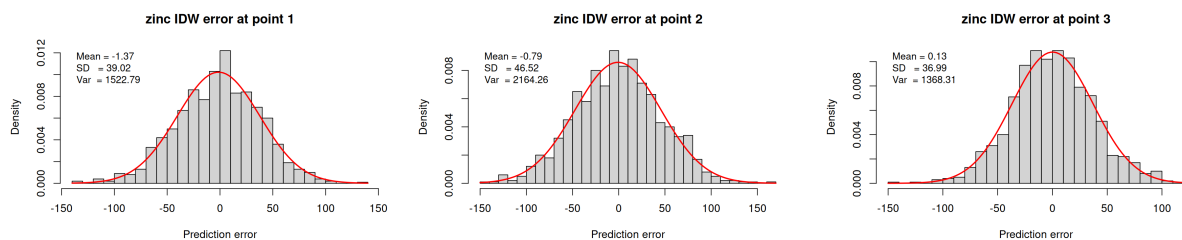


```
summary_stats <- function(df, metal) {
  vals <- df[[paste0(metal, "_error")]]
  c(
    Mean = mean(vals, na.rm = TRUE),
    SD = sd(vals, na.rm = TRUE),
    Var = var(vals, na.rm = TRUE)
  )
}

rbind(
  P1 = summary_stats(df_errors_p1, "cd"),
  P2 = summary_stats(df_errors_p2, "cd"),
  P3 = summary_stats(df_errors_p3, "cd")
)
```

```
##           Mean          SD          Var
## P1 -0.013114502 0.3746034 0.1403277
## P2 -0.007623008 0.4465868 0.1994398
## P3  0.001237558 0.3550939 0.1260917
```

Zinc (Zn)



```
rbind(
  P1 = summary_stats(df_errors_p1, "zinc"),
```

```

P2 = summary_stats(df_errors_p2, "zinc"),
P3 = summary_stats(df_errors_p3, "zinc")
)

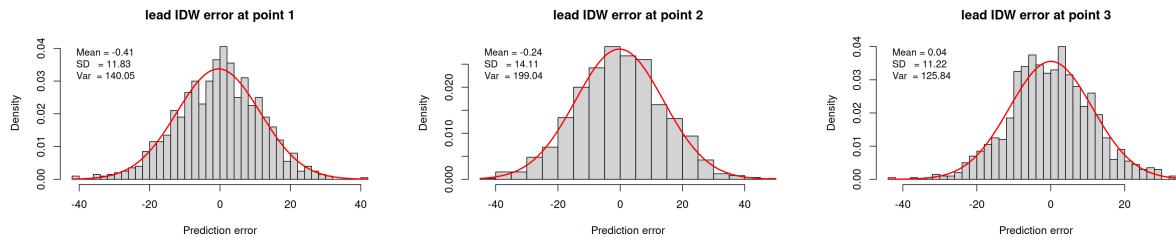
```

```

##           Mean      SD      Var
## P1 -1.3661570 39.02299 1522.794
## P2 -0.7941000 46.52161 2164.260
## P3  0.1289182 36.99066 1368.309

```

Lead (Pb)



```

rbind(
  P1 = summary_stats(df_errors_p1, "lead"),
  P2 = summary_stats(df_errors_p2, "lead"),
  P3 = summary_stats(df_errors_p3, "lead")
)

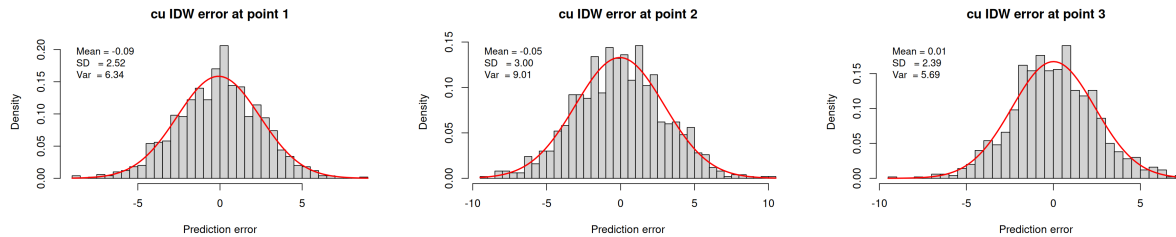
```

```

##           Mean      SD      Var
## P1 -0.41430545 11.83425 140.0494
## P2 -0.24082148 14.10830 199.0442
## P3  0.03909619 11.21791 125.8416

```

Copper (Cu)



```

rbind(
  P1 = summary_stats(df_errors_p1, "cu"),
  P2 = summary_stats(df_errors_p2, "cu"),
  P3 = summary_stats(df_errors_p3, "cu")
)

```

##		Mean	SD	Var
##	P1	-0.088132672	2.517427	6.337438
##	P2	-0.051228484	3.001173	9.007040
##	P3	0.008316693	2.386318	5.694515

Discussion

The results indicate that IDW interpolation exhibits consistent bias at certain spatial locations. While the variance of error remains low across realizations, the mean error suggests systematic deviation.

Differences in standard deviation across metals are primarily due to differences in concentration scale and the proportional noise model used.

Conclusion

IDW interpolation provides stable but potentially biased estimates. The database-driven workflow enables efficient and reproducible analysis.

Future improvements may include:

- Comparing IDW with kriging
- Normalizing error metrics across metals
- Increasing spatial sampling density